

Algorytmy geometryczne, ćwiczenie 4

1. Dane techniczne

- Procesor: Intel Core i7-7700HQ 2.80GHz
- RAM: 16GB 2133 MHz
- System operacyjny: Windows 10 Home x64
- Środowisko: Visual Studio Code
- Język: Python 3.13.5
- Biblioteki: numpy, pandas, random, matplotlib, heapq, sortedcontainers. Do graficznej wizualizacji wykorzystano narzędzie koła naukowego BIT.

2. Cel ćwiczenia

Celem ćwiczenia było poznanie i implementacja algorytmu zmiatania (sweep-line algorithm) w dwóch wariantach: sprawdzania czy w zbiorze odcinków występuje przynajmniej jedno przecięcie oraz wyznaczenia wszystkich punktów przecięć. Ćwiczenie obejmowało również analizę działania algorytmu na różnych zestawach danych oraz wizualizację wyników.

3. Wstęp teoretyczny

3.1. Zagadnienia

Dla danego zbioru odcinków $S = \{s_1, s_2, \dots, s_n\} \in \mathbb{R}^2$ należy:

- Sprawdzić czy istnieje para (s_i, s_j) taka, że $i \neq j$ oraz $s_i \cap s_j \neq \emptyset$.
- Znaleźć wszystkie pary (s_i, s_j) takie, że $i \neq j$ oraz $s_i \cap s_j \neq \emptyset$, wraz z punktami przecięć.

3.2. Założenia

- Żadne dwa punkty (końce odcinków, przecięcia) nie mają tej samej współrzędnej x .
- Każdy punkt przecięcia należy tylko do dwóch odcinków z S .

3.3. Algorytm zmiatania

Algorytm zmiatania to efektywna metoda wykrywania przecięć odcinków na płaszczyźnie, która wykorzystuje wirtualną prostą (miotłę) przesuwającą się wzdłuż osi OX . Prosta ta zatrzymuje się w punktach zdarzeń w celu aktualizacji stanu. Metoda opiera się na dwóch strukturach: strukturze stanu (T), przechowującej aktywne odcinki uporządkowane według współrzędnej y , oraz strukturze zdarzeń (Q), zawierającej posortowane rosnąco względem x punkty zdarzeń. Dzięki temu algorytm sprawdza kolizje wyłącznie dla par odcinków sąsiadujących ze sobą na miotle, co znacząco redukuje liczbę obliczeń względem metody naiwnej.

4. Implementacja

4.1. Klasy pomocnicze

Do reprezentacji obiektów geometrycznych użyto klas Point i Segment:

- Point: zawiera współrzędne punktu (float64), typ zdarzenia (L - początek, R - koniec, I - przecięcie) oraz indeks odcinka.
- Segment: zawiera dwa obiekty Point (p_1, p_2), indeks w S oraz współczynniki prostej (a, b) niezbędne do obliczania współrzędnej y na miotle.

4.2. Sprawdzanie występowania przecięcia

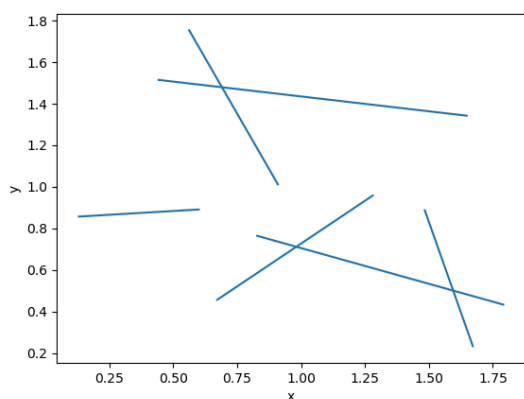
- Struktura stanu T : SortedSet z biblioteki sortedcontainers. Zapewnia operacje wstawiania, usuwania i wyszukiwania w złożoności $O(\log n)$.

- Struktura zdarzeń Q : Lista wbudowana w Python. Sortowana raz na początku ($O(n \log n)$), ponieważ interesuje nas tylko pierwsze przecięcie.
- Funkcja `check_intersection(s1, s2)`: Weryfikuje czy dwa odcinki się przecinają. Odrzuca przypadki trywialne, wylicza punkt przecięcia i sprawdza jego przynależność do odcinków.
- Funkcja `is_intersection(sections)`: Iteruje po zdarzeniach. Dla typu L dodaje odcinek do T i sprawdza sąsiadów. Dla typu R usuwa odcinek i sprawdza nowych sąsiadów. Zwraca True natychmiast po wykryciu kolizji.

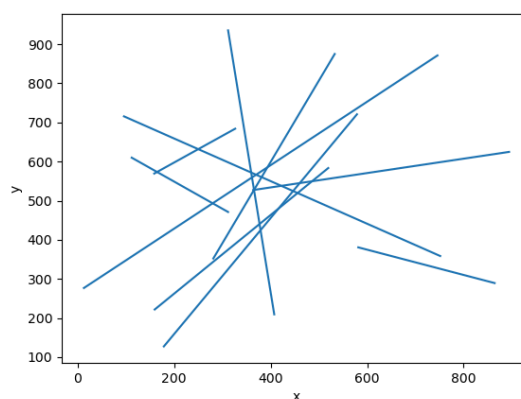
4.3. Znajdowanie wszystkich przecięć

- Struktura stanu T : Analogicznie jak w sekcji 4.2 (SortedSet).
- Struktura zdarzeń Q : Kopiec binarny (heapq). Pozwala na dynamiczne dodawanie nowych punktów przecięć w czasie $O(\log n)$.
- Optymalizacja: Użyto zbioru `checked_pairs` do przechowywania par już sprawdzonych, co zapobiega pętlom i nadmiarowym obliczeniom.
- Funkcja `find_intersection(sections)`: Rozbudowana o obsługę zdarzenia typu I. Po wykryciu przecięcia punkt trafia do wyników, a w strukturze T następuje zamiana kolejności przecinających się odcinków i sprawdzenie nowych sąsiadów.

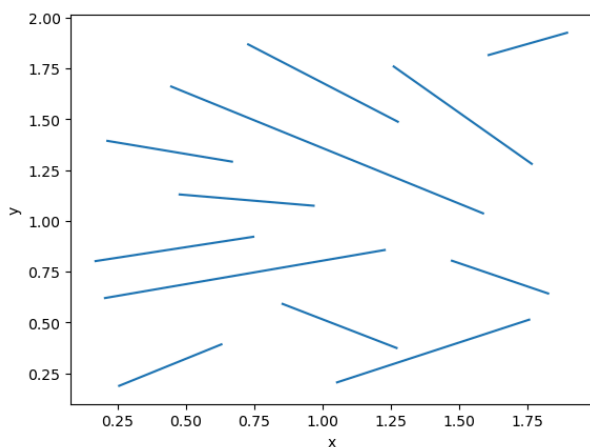
5. Zbiory danych



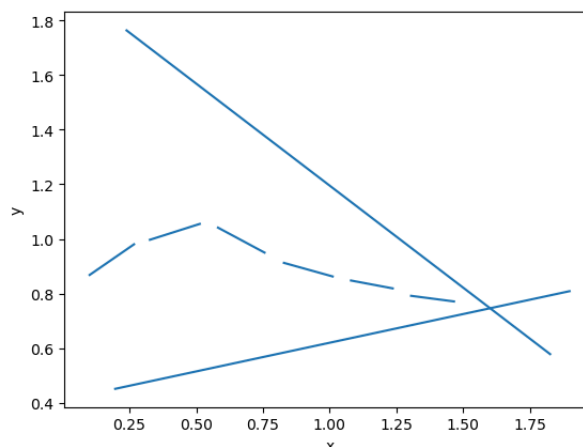
Rys. 1: Zbiór A, 6 odcinków, 3 przecięcia (test podstawowy).



Rys. 2: Zbiór B, 10 losowych odcinków [0, 1000] (test ogólny).



Rys. 3: Zbiór C, 12 odcinków rozłącznych (brak przecięć).



Rys. 4: Zbiór D, 8 odcinków z jedną parą przecinającą się.

Zbiory zostały dobrane w taki sposób, żeby sprawdzały działanie algorytmów na różnych zestawach danych. Zbiór A to podstawowy przypadek sprawdzający działanie na prostych danych. Zbiór B to

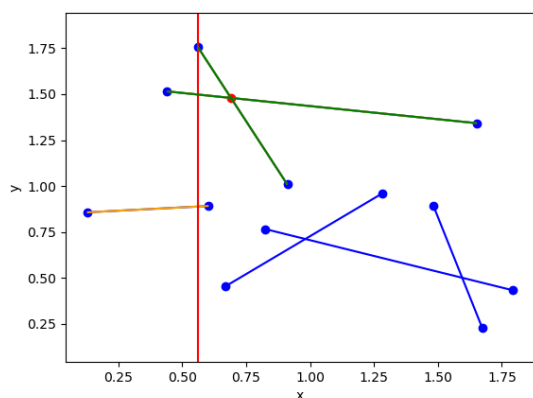
bardziej komplikowany zestaw sprawdzający działanie na losowych zbiorach. Zbiór C to przypadek sprawdzający wystąpienie zjawiska wykrycia przecięcia tam gdzie go nie ma. Zbiór D jest tak skonstruowany żeby jedyne przecięcie jakie się w nim znajduje było wykrywane wielokrotnie.

6. Analiza wyników

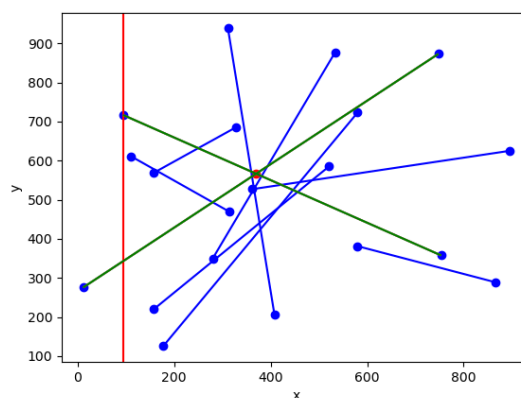
6.1 Algorytm sprawdzający istnienie przecięcia

Zbiór	A	B	C	D
Wynik	True	True	False	True

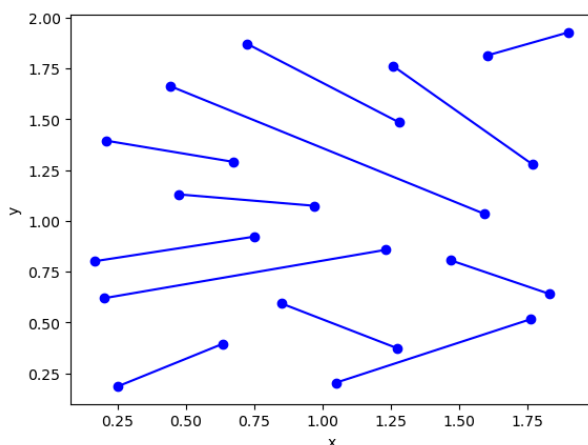
Tabela 1: Wyniki działania algorytmu na poszczególnych zbiorach.



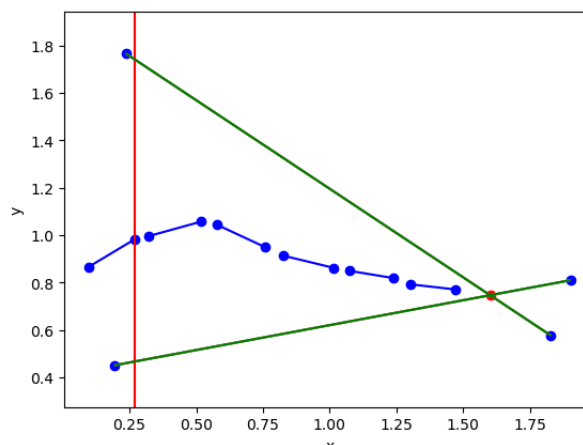
Rys 5. Wykrycie pierwszego przecięcia w zbiorze A.



Rys 6. Wykrycie pierwszego przecięcia w zbiorze B.



Rys 7. Zakończenie bez wykrycia przecięć (Zbiór C).



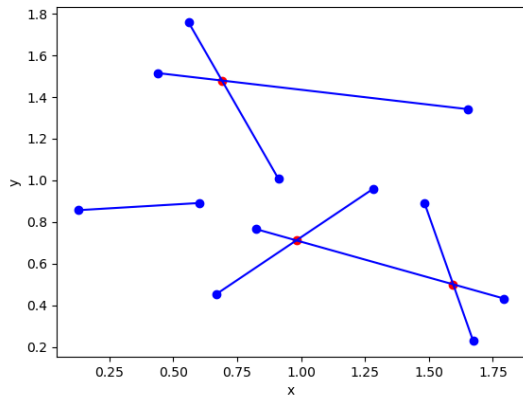
Rys 8. Wykrycie przecięcia w zbiorze D.

Z danych w Tabeli 1. wynika, że Algorytm poprawnie zidentyfikował obecność przecięć w zbiorach A, B i D oraz ich brak w zbiorze C. Dodatkowo na Rys. 5-8 przedstawiono moment zakończenia Algorytmu, na niebiesko punkty i odcinki wejściowe, na żółto odcinki w T, na zielono obecnie przetwarzana para odcinków, na czerwono znaleziony punkt i pozycja miotły.

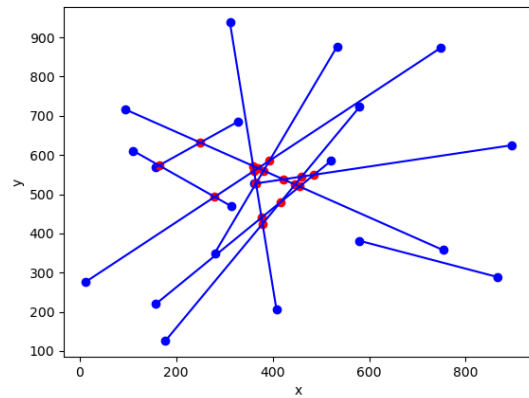
6.2 Algorytm znajdujący wszystkie punkty przecięcia

Zbiór	A	B	C	D
liczba odcinków	6	10	12	8
Liczba przecięć	3	19	0	1

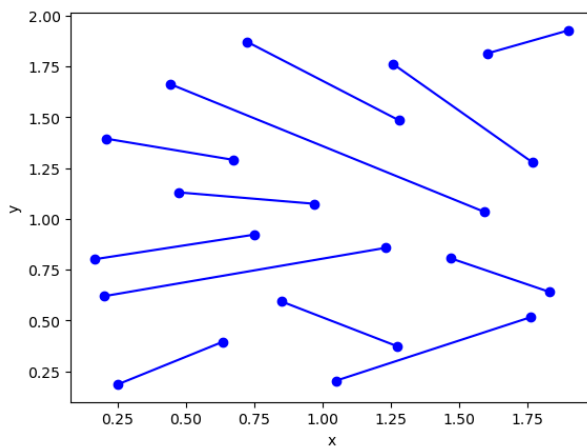
Tabela 2: Zestawienie liczby wykrytych przecięć i ilości odcinków w poszczególnych zbiorach.



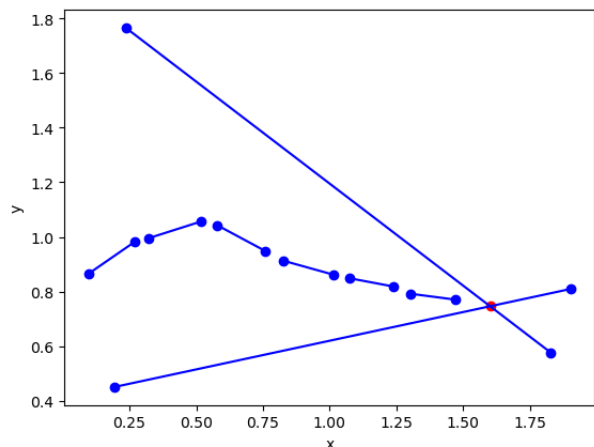
Rys 9. Wszystkie przecięcia w zbiorze A.



Rys 10. Wszystkie przecięcia w zbiorze B.



Rys 11. Brak przecięć w zbiorze C.



Rys 12. Jedno przecięcie w zbiorze D.

Algorytm poprawnie wyznaczył wszystkie punkty przecięć. Dane w Tabeli 2. zgadzają się z wizualizacją wyników na Rys. 9-12, Na niebiesko punkty i odcinki wyjściowe i na czerwono punkty przecięcia.

7. Wnioski

Na podstawie przeprowadzonych testów można sformułować następujące wnioski:

- **Poprawność:** Zaimplementowany algorytm działa poprawnie, identyfikuje przecięcia oraz radzi sobie z ich brakiem (brak false positives w zbiorze C).
- **Efektywność:** Algorytm zmiatania jest znacznie wydajniejszy od metody naiwnej ($O(n^2)$). Dzięki wykorzystaniu sąsiedztwa w strukturze T , liczba testów przecięcia jest zminimalizowana. Złożoność wynosi $O((n + k) \log n)$.
- **Struktury danych:** Kluczowe jest użycie SortedSet (czas operacji $O(\log n)$) oraz kolejki priorytetowej heapq do zachowania porządku zadrzeń przy dynamicznym dodawaniu przecięć.
- **Obsługa stanu:** Mechanizm zamiany kolejności odcinków w strukturze T po wystąpieniu przecięcia jest krytyczny dla wykrywania kolejnych kolizji z nowymi sąsiadami.
- **Optymalizacja:** Istotnym elementem jest zbiór checked_pairs, który zapobiega wielokrotnemu przetwarzaniu tych samych par odcinków.